



PHASE 4 • BUILD

Group Project Kickoff.

Today the learning stops and the building begins. Teams formed, leaders chosen, projects picked. Seven days of building. One demo. Let's go.

Zlaark

SESSION CARD

PROGRAM **Python Internship**

LED BY **Zlaark**

SESSION **16 of 25**

HINDU COLLEGE AMRITSAR • BCA III

12 AUGUST 2026

WEDNESDAY • BUILD WEEK BEGINS

How we'll spend our 90 minutes today.

Today is about setup and decisions — teams, projects, repos, plans. By the end, every team has a name, a leader, a project, and a plan.

00:00 – 00:10

01

🔄 Phase 3 Recap

Quick reminder of what you learned. The toolkit you'll use.

00:10 – 00:30

02

👥 Team Formation

Leaders volunteer. Leaders pick one person. I assign 1-2 to balance. Teams name themselves.

00:30 – 00:50

03

📋 Project Selection

20 projects across 3 tracks. Each team picks one. Match your ambition.

00:50 – 01:00

04

☁️ GitHub Setup

One repo per team. Everyone added. First commit today.

01:00 – 01:20

05

☰ Project Planning

Lock the scope. Write the plan. Set sprint goals. Team leader drives.

01:20 – 01:30

06

🚀 Sprint Preview + Close

What happens tomorrow (Sprint 1). Team huddle.

→ Flow: recap → form → pick → setup → plan → ship

👥 Result: a team that's ready to build tomorrow

Seven days. Five sprints. One demo.

Each sprint has a goal and a checkpoint. Your team leader drives the sprint. I move between teams every session.

Today is Day 16



After Day 22: Day 23 dress rehearsal, Day 24 polish buffer, Day 25 Demo Day.

THE RHYTHM Build (1-5) → Ship (Deploy) → Rehearse (Dress) → Present (Demo)

Same rhythm every software team uses.

You already know your team. Now let's get to work.

Groups were formed during the learning phase — you've been working alongside your teammates for 15 days. Today we make it official: confirm your team, pick a leader, name your team, and start building.

✓ DONE DURING LEARNING

5 / 5 ✓

- ✓ **Teams are formed**
I assigned groups during the learning phase based on what I saw over 15 days.
- ✓ **You know your teammates**
You've been sitting together, working together, helping each other.
- ✓ **Mixed by ability**
Every team has a mix of strong and growing students. This is intentional.
- ✓ **You know the tools**
Python, Streamlit, SQLite, regex, APIs, TF-IDF — everyone has the same foundation.
- ✓ **You know Git**
Day 3. Commits, push, pull, merge conflicts. You'll use this daily now.

No surprises today. The team you've been with IS your project team.

BEFORE YOU PICK A PROJECT

5 STEPS

- 1 Confirm your team leader**
One person takes responsibility for driving the project. Not the boss — the coordinator. The leader ensures everyone contributes and checkpoints are hit.
- 2 Name your team**
"Code Warriors", "Python Pioneers", "Debug Squad". The name goes in your GitHub repo: `group-N-teamname`.
- 3 Set up communication**
WhatsApp/Discord group with all members + me (instructor). This is how you coordinate: "I'm pushing the form code now."
- 4 Pick a project**
20 projects across 3 tracks (next slides). Match your team's ambition.
- 5 Create your GitHub repo**
One repo per team. Everyone is a collaborator. First commit today.

THE LEADER — Drives the project, not commands. Makes final calls on scope and features. Communicates with me if the team is stuck. Presents first on Demo Day. Every team member still codes and presents.

NO RESHUFFLING

Teams are locked. If there's a real problem with a teammate, tell me privately. I'll handle it. Don't take it into the project.

This internship is worth 50 marks. Here's how.

The university assesses this internship as an End-Semester Examination. Three components: a written report (30 marks), a seminar presentation (10 marks), and a Q/A session (10 marks). Start preparing NOW — not on the last day.

MARKS BREAKDOWN

ESE • 50 MARKS

COMPONENT	MARKS	WEIGHT
Internship Report	30	60%
Seminar Presentation	10	20%
Q/A Session	10	20%
Total	50	100%

KEY INSIGHT — The report is 60% of your grade. Start writing it from Day 1 — take daily notes. The daily notes become 40% of the report. Students who fail usually wrote their entire report in 2 days.

WHAT EACH COMPONENT MEANS

ALL INDIVIDUAL



REPORT • 30 MARKS

A written document (25-35 pages) covering: what you learned, what you built, your role, challenges, code samples, learning outcomes.

INDIVIDUAL — each student writes their OWN report, even for team projects.

START
Day 1
SUBMIT
Demo Day



PRESENTATION • 10 MARKS

A 5-8 minute oral presentation with PowerPoint slides. Show your project working live. Explain YOUR role.

INDIVIDUAL — each student presents separately.

PREP
Day 23
PRESENT
Day 25



Q/A SESSION • 10 MARKS

Examiners ask you questions after your presentation. They test if you understand your code, your project, and the concepts.

INDIVIDUAL — they ask YOU, not your team.

PREP
Night B4
PRACTICE
w/ Friend

TEAM WORK IS OK

The project is built in teams. The assessment is INDIVIDUAL. Your report describes the SAME project, but from YOUR perspective: your role, your code, your challenges.

What goes in the report. What goes in the PPT. When to start.

A full assessment guide PDF is in the shared folder — download it for detailed templates. This slide is the summary.

REPORT • 25-35 PAGES

30 MARKS

KEY SECTIONS

- 01 Cover + Certificate + TOC
- 02 Introduction (what the internship was about)
- 03 Organization Profile (about Zlaark)
- 04 Technologies Used (Python, Streamlit, SQLite, etc.)
- 05 **Daily Work Log (8-10 pages)** — one paragraph per day, Days 1-25. THIS is why you take daily notes.
- 06 Project Description (what you built, your role, screenshots)
- 07 Code Samples (3-5 key functions with explanations)
- 08 Challenges & Solutions (3-5 real problems you solved)
- 09 Learning Outcomes (what you learned)
- 10 Conclusion + References

START NOW — Take 2-3 sentences of notes every day. These become 60% of your report.

PRESENTATION • 5-8 MIN

10 MARKS

12-SLIDE STRUCTURE

- 01 Title (project, team, your role)
- 02 Problem (what does your app solve?)
- 03 Solution (one sentence + screenshot)
- 04 Tech Stack (Python, Streamlit, etc.)
- 05 Architecture (how it works)
- 06 **LIVE DEMO** — show the app running (most important!)
- 07 AI Component (if TF-IDF/AI API — explain simply)
- 08 Challenges (1-2 you faced)
- 09 Your Role (what YOU did)
- 10 Future Improvements
- 11 GitHub Link
- 12 Thank You + Q&A

LIVE DEMO > SCREENSHOTS — If your app works, SHOW it. Run it live. That's worth more than any slide.

TIMELINE

WHEN TO START

VERTICAL TIMELINE

- **DAYS 1-15**
Take daily notes (2-3 sentences/day) → becomes Daily Log
- **DAY 15**
Write Learning Outcomes
- **DAYS 16-22**
Document project as you build → screenshots, challenges
- **DAY 22-23**
Write Technologies section + draft PPT
- **DAY 23**
Practice presentation 3+ times with timer
- **DAY 24**
Finalize report. Print. Practice Q/A.
- **DAY 25**
Present + Q/A + submit report. Done.

FULL GUIDE — A detailed assessment guide PDF is in the shared folder. Download it. Has: 15-section template, 24 common Q/A questions, formatting guidelines, checklists.

THE BIGGEST MISTAKE Students who score low wrote their report in 2 days. It shows: thin daily log, no real challenges, generic descriptions. Start daily notes from Day 1 — that's 60% of the report done before you even start the project.

20 projects. AI-first. Modern. Every one is something you'd use.

Track A uses TF-IDF + scikit-learn (no external API — works offline). Track B calls Groq free API (14,400 req/day, no credit card). Both are demo-worthy.

20 of 30 projects

TRACK A • FOUNDATION • PROJECTS 1-10

AI WITH WHAT YOU KNOW — NO API NEEDED

- 1 Smart Quiz Generator**
Paste notes, **TF-IDF** finds key sentences, auto-generates 5 MCQ questions. Exam prep tool.
- 2 Campus FAQ Chatbot**
Type a question about college, TF-IDF matches to stored answers. ChatGPT-style UI, **powered by math**.
- 3 AI Resume Analyzer**
Upload resume PDF, extract top 15 keywords by TF-IDF weight, show keyword cloud. Compare to a job description.
- 4 Sentiment Analyzer for Reviews**
Paste product reviews, classify positive/negative/neutral using **scikit-learn** Naive Bayes. Show breakdown chart.
- 5 Content-Based Movie Recommender**
Pick a movie, get 5 similar recommendations based on genre + description TF-IDF. **Real recommendation system**.
- 6 Smart Notes Summarizer**
Paste long lecture notes, get top 5 most important sentences by TF-IDF + key terms + reading time.
- 7 Fake News Detector**
Paste a news article, classify as "likely real" or "likely fake" using Naive Bayes. **I provide the dataset**.
- 8 Live Data Dashboard**
Call a free API (Open-Meteo weather, currency rates), display on a dashboard with charts. **Day 12 API + Day 7 charts**.
- 9 Web Scraper + Display Tool**
Scrape news headlines / prices / weather, clean with regex, display in Streamlit. **Day 11 regex + Day 12 requests**.
- 10 PDF Toolkit**
Upload multiple PDFs, merge / split / watermark. Pure Python file handling + **PyPDF2**. **Practical, no AI needed**.

Uses: TF-IDF, scikit-learn, PyPDF2, requests, regex, Streamlit (Days 4-14). No external AI API needed — works offline.

TRACK B • INTERMEDIATE • PROJECTS 11-20

AI WITH FREE GROQ API

- 11 AI Study Buddy**
Paste a topic, **Groq** generates: a summary, 3 key points, and 3 practice questions. Like a personal tutor.
- 12 Resume Tailor AI**
Upload resume + paste a job description. Groq suggests 3 bullet-point improvements to match the job better. **Career tool**.
- 13 Multi-Language Translator**
Paste text, select language (Hindi, Punjabi, French, Spanish), Groq translates. Shows original + translation side by side.
- 14 AI Email Drafter**
Select type (leave application, internship inquiry, professor follow-up), fill details, Groq generates a formal email.
- 15 Smart Document Q&A (RAG)**
Upload a PDF, ask questions. TF-IDF finds the relevant paragraph, Groq generates a natural-language answer. **The RAG pattern**.
- 16 AI Code Explainer**
Paste Python code, Groq explains what it does line by line in simple English. **Like having a senior dev explain code**.
- 17 Action Item Extractor**
Paste meeting/class notes, Groq extracts: action items, deadlines, and key decisions. Structured output from unstructured text.
- 18 AI Meme Generator**
Select an image template, Groq generates a funny caption based on a topic. **Pure fun + real API usage. Students love this**.
- 19 Smart File Organizer**
Upload multiple files, TF-IDF analyzes content, auto-categorizes by topic. **Like a mini Google Drive auto-sort**.
- 20 AI-Powered Expense Categorizer**
Type "bought coffee at Starbucks", Groq auto-categorizes (Food → Coffee) and suggests a budget. **Smart finance tracker**.

Adds: Groq API (free, 14,400 req/day, no credit card). Same `requests.get()` pattern from Day 12.

BOTH TRACKS Track A = no API, works offline. Track B = calls Groq free API. Both demo-worthy. Track C (advanced) on next slide →

10 advanced projects. Multi-API. Real-world. Deployable.

These combine TF-IDF + scikit-learn + Groq + real data + complex UI. 2–3 could be deployed for actual college use. Portfolio pieces that make employers say: “A student built this?”

🏆 Track C • 10 projects

C TRACK C • ADVANCED REAL-WORLD AI • PROJECTS 21–30

🏆 PORTFOLIO-GRADE • DEPLOYABLE

— PROJECTS 21 – 25

- 21

 **AI Interview Coach**
Upload resume + paste job description. **Groq** generates 5 likely interview questions based on the job. You answer. Groq scores your answers (relevance, clarity). **Full interview prep loop.**
- 22

 **AI-Powered News Digest**
Fetch news from free RSS feeds, Groq summarizes each article in 2 sentences, TF-IDF categorizes by topic, show a personalized feed. **Like In Shorts — but you built it.**
- 23

 **Smart Assignment Checker**
Upload an assignment + rubric. TF-IDF checks content coverage. Groq checks grammar + gives feedback. Shows a score + improvement suggestions. **Professors would use this.**
- 24

 **AI Content Detector**
Paste text, classify as “AI-generated” or “human-written” using a trained **scikit-learn** model on a labeled dataset. The tool professors actually want. **I provide the dataset.**
- 25

 **Personal AI Tutor**
Pick a subject + topic, Groq generates a mini-lesson (explanation + example + quiz). Tracks your progress in **SQLite**. **Like Duolingo for college subjects.**

— PROJECTS 26 – 30

- 26

 **AI Resume Builder + ATS Score**
Fill in details (Streamlit form), generate a formatted resume, score it against a job description (TF-IDF + Groq suggestions for missing keywords). **End-to-end career tool.**
- 27

 **Smart Chatbot with Memory**
A chatbot that remembers conversation history (**session_state** + SQLite), uses Groq for responses, answers follow-up questions. **Like a simplified ChatGPT with persistence.**
- 28

 **AI Feedback Analyzer for College**
Collect feedback (rating + text), analyze sentiment per response (scikit-learn), generate an overall summary report (Groq), show trends on a dashboard. **Deployable for course/professor feedback.**
- 29

 **Document Intelligence Dashboard**
Upload any PDF/Word, get: summary (Groq), key terms (TF-IDF), sentiment (**scikit-learn**), translation (Groq), reading time, difficulty score. **All-in-one analysis dashboard.**
- 30

 **Multilingual AI Assistant for Campus**
Ask questions in Hindi, Punjabi, or English. Groq translates + answers + matches to campus info (TF-IDF). **A chatbot that serves ALL students. Deployable for the college.**

REAL-WORLD IMPACT

23 (Assignment Checker), 24 (AI Detector), 28 (Feedback Analyzer), and 30 (Multilingual Assistant) solve real problems. **These aren’t demos — they’re tools people would use.**

20 minutes. Read. Vote. Discuss. Decide.

The team leader drives this process. Everyone has a say. The leader makes the final call.

 Leader-driven • 20 min

THE 4-STEP PROCESS

How a team picks their project

- 1 Read the list** 3 min
Everyone reads all 20 project descriptions. No discussion yet.
- 2 Vote individually** 2 min
Each member writes their top 3 on paper. No groupthink.
- 3 Discuss** 10 min
Share your top 3. Find overlap. The leader facilitates.
- 4 Decide + tell me** 5 min
Leader locks the choice. Tells me. I approve the scope.

If you can't agree — the leader decides. That's why they're the leader.

HOW TO CHOOSE

Five questions every team should ask

- What excites the team?**
Excitement drives quality. Pick something you'd actually use.
- Can you build the core in 3 sprints?**
If the core takes more than Sprints 1-3, scale down.
- Does everyone understand it?**
If half the team doesn't get it, pick something else.
- Can you demo it in 2 minutes?**
Imagine Demo Day. 2 minutes. Is it clear?
- Are you reaching?**
Track C is ambitious. If your team is ready, go for it. If unsure, Track A or B is fine.

SCOPE DISCIPLINE A working app with **3 features** beats a broken app with **7**. Pick a project you can FINISH. You can always add features in Sprint 5 if you're ahead.

One repo per team. Everyone commits. Today.

The team leader creates the repo. Everyone else is added as a collaborator. First commit today.



github_setup.sh

BASH

```
1 # Step 1: LEADER creates the repo
2 # github.com → New repository
3 # Name: group-N-teamname (e.g. group-3-code-warriors)
4 # Set to PUBLIC. Tick "Add a README".
5
6 # Step 2: Add collaborators
7 # Settings → Collaborators → Add people
8 # Add every team member by GitHub username
9 # They get an email – they MUST accept
10
11 # Step 3: Everyone clones
12 git clone https://github.com/LEADERNAME/group-N-teamname.git
13 cd group-N-teamname
14
15 # Step 4: Add .gitignore (see right)
16
17 # Step 5: First commit
18 git add .gitignore README.md
19 git commit -m "Initial commit – [Team Name] project kickoff"
20 git push origin main
21
22 # Step 6: Everyone tests
23 # Each member clones, makes a test commit, pushes.
```

.GITIGNORE + GIT RULES

.gitignore

```
__pycache__/
*.pyc
.venv/
.DS_Store
*.db
*.sqlite3
.env
data/
```

- ✓ **Commit often** — every feature, every fix
- ✓ **Meaningful messages:** "Add expense form" not "update"
- ✓ **Pull before you push** — avoid conflicts
- ✓ **Never commit** .venv or .db files
- ✓ **If merge conflict** — call me. We fix together.
- ✓ **Communicate before pushing** — tell the team

DAY 3 REVISITED

Everything from Day 3 applies now. The difference: you're in a **TEAM**. Communication is as important as code.

Scope it. Write it down. Stick to it.

Before any code, lock what you're building. The leader drives this. Everyone contributes ideas. The plan goes in the README.

PROJECT PLAN TEMPLATE

```
## Project: [NAME] ## Team: [TEAM NAME] ## Leader: [NAME] ## Members:
[NAMES] ### What it does (1 sentence): [e.g. "Upload a resume PDF and
compare it to a job description."] ### Core features (MUST have): 1.
[...] 2. [...] 3. [...] ### Nice-to-have (IF time permits): - [...] -
[...] ### Tech stack: - Streamlit (UI) - [SQLite/CSV] (storage) -
[scikit-learn/pandas] (logic) ### Sprint 1 goal: ["App runs, has
title + navigation"]
```

SPRINT BREAKDOWN

SPRINT	GOAL	CHECKPOINT
1: Skeleton	App runs, navigation	Running Streamlit app
2: Core Logic	Main feature works	Feature works on one example
3: Data	Save/load persists	Data survives restart
4: Polish	Handles bad input	Can't crash it
5: Freeze	Full run-through	Demo-ready

SCOPE RULE — Core features are MUST. Nice-to-haves are BONUS. Behind on Day 20? **DROP** the nice-to-haves. Ship the core.

WRITE IT IN THE README

Don't keep the plan in your head. Write it in README.md. Everyone can read it. Everyone knows what you're building.

Five criteria. Transparent from Day 1.

You know exactly what I'm looking for. No surprises on Demo Day.

CRITERION 01

WORKING APP 30%

Does the app work? Can you demo it start to finish without crashing?

MEASURED BY: live demo on Day 25.

"A working app with 3 features beats a broken app with 7."

CRITERION 02

CODE QUALITY 20%

Readable code? Named functions? Comments? No 200-line functions?

MEASURED BY: GitHub repo review.

"I read your code. Not just the demo."

CRITERION 03

GIT DISCIPLINE 15%

Regular commits with meaningful messages. Not one big upload.

MEASURED BY: commit history on GitHub.

"I look at your commit graph. Green squares tell the story."

CRITERION 04

SCOPE MATCH 15%

Does the finished app match the scope you locked today?

MEASURED BY: your plan vs your demo.

"Planned 3, shipped 3 = full marks. Planned 5, shipped 2 = scope mismatch."

CRITERION 05

PRESENTATION 20%

Can every student explain their part? Answer questions?

MEASURED BY: Demo Day Q&A.

"Every student presents. Every student answers at least one question."

THE BIGGEST MISTAKE Scope creep. Plan 7 features, build 3, demo broken. Lock scope today. Ship the core.

Five rules. Five anti-patterns. Read both.

Most projects fail because of people problems, not code. Here's how to avoid the traps.

✓ THE 5 RULES

- ✓ **Communicate before you push**
Tell the team: "I'm pushing now." No surprise merges.
- ✓ **Commit often, commit small**
One feature per commit. Meaningful messages.
- ✓ **Ask for help early**
Stuck 30 min? Ask the team. Stuck 1 hour? Ask me.
- ✓ **Everyone codes, everyone presents**
No one just watches. No one does everything.
- ✓ **Be honest about gaps**
"I don't understand the SQL part" is better than pretending and breaking it.

⚠ THE 5 ANTI-PATTERNS

- ✗ **The hero coder**
One person does everything. Others can't explain it on Demo Day.
- ✗ **The ghost**
Someone never shows up. Tell me privately — I'll handle it.
- ✗ **The scope creeper**
"Let's add authentication! And a chat! And..." No. Lock scope. Ship core.
- ✗ **The last-minute push**
All commits on Day 21 at midnight. I notice. Git discipline = 15%.
- ✗ **The silent conflict**
Two people disagree. Nobody talks. Both push. Merge conflict. Drama. Communicate FIRST.

THE GOLDEN RULE If something is wrong — dynamics, scope, a teammate not contributing — tell me EARLY. Day 17 is fixable. Day 24 is not.

Everything you need. You already learned it.

15 days of skills. Here's what each one is FOR — so you know what to use when.

DAYS 1-2 01 Python Functions, files, conditionals. The foundation.	DAYS 4-5 02 Streamlit The UI. <code>file_uploader</code>, <code>text_area</code>, <code>metric</code>, <code>columns</code>, <code>sidebar</code>.	DAY 6 03 pandas DataFrames. Load CSV, filter, sort, groupby.
DAY 7 04 Charts <code>st.bar_chart</code>, <code>st.line_chart</code>, Matplotlib.	DAYS 9-10 05 SQLite <code>CREATE TABLE</code>, <code>INSERT</code>, <code>SELECT</code>. The helper-function pattern.	DAY 11 06 Regex <code>re.findall</code>, <code>re.sub</code>. Extract patterns, clean text.
DAY 11 07 PDF/Word PyPDF2, <code>python-docx</code>. Extract text from uploaded files.	DAY 12 08 APIs <code>requests.get</code>, JSON. Live data from the internet.	DAYS 13-14 09 TF-IDF + AI <code>scikit-learn</code>. Text similarity, match scores.

CHEATSHEETS

All cheatsheets from Days 6-13 are still in your notes. Bookmark them. You don't need to memorize — you need to know WHERE to look.

Tomorrow: Sprint 1 — Skeleton.

Get something running. Even if it does nothing useful yet. A Streamlit app with your project's title, navigation, and placeholder screens.

WHAT TO BUILD

- ✓ `st.title("[Your Project Name]")`
- ✓ Navigation — tabs or sidebar for multiple views
- ✓ Placeholder screens — `st.write("[Feature] goes here")`
- ✓ A sidebar with team name + members
- ✓ Run it — `streamlit run app.py`

DON'T BUILD YET — No database, no logic, no API. Just the **SKELETON**. Fill it in Sprints 2-5.

CHECKPOINT

By end of Day 17: **6 ITEMS**

- ✓ A running Streamlit app
- ✓ Project title on the page
- ✓ Navigation between views (if applicable)
- ✓ Placeholder text on each view
- ✓ At least ONE commit from EACH member
- ✓ Everyone has cloned + can run the repo

IF DONE EARLY — Start Sprint 2 (Core Logic). Don't wait for me.

TOMORROW'S MINDSET

"Something that runs" beats "something perfect that doesn't." Ship the skeleton. Make it ugly. Make it work.

Today's checklist. Do all of these before you leave.

If every team completes this checklist, Day 16 is a success.

01



Team formed

You know your team. 3-5 members.

02



Team named

You picked a team name.

03



Leader confirmed

The team leader is decided and named in the README.

04



Communication channel set up

WhatsApp/Discord group with all members + instructor.

05



Project picked

One of the 20 projects (or your own, approved).

06



Scope locked

Core features + nice-to-haves written in README.

07



GitHub repo created

Leader created it. Name: `group-N-teamname`.

08



Everyone added as collaborator

All members can push. Tested with a commit.

09



`.gitignore` added

No `.venv`, no `.db` files.

10



First commit pushed

README with project name + team + plan.

IF YOU'RE STUCK

raise your hand. I'll come to your team. Fix it now, not on Day 17.

Seven days of learning. Now you use it.

Quick reminder of what you learned in Phase 3. This is the toolkit for your project.

DAY	DAY	DAY	DAY	DAY	DAY	DAY
9	10	11	12	13	14	15
Saving Data Part 1 JSON, databases, SQL, SELECT, CREATE TABLE	Building with SQLite SQLite CRUD, Streamlit pattern, expense tracker	Working With Text Regex, PDF extraction, Word docs	Calling Real APIs requests, JSON, caching, weather dashboard	A First Taste of AI TF-IDF, cosine similarity, resume matcher	Resume Matcher Build Full app: PDF + TF- IDF + Streamlit	Recap & Doubt Clearing Identify, fix, verify. Ready to build.



The learning is done. The building begins.

15 days of skills. 10 days of building. 1 demo. You have everything you need. Now go build something real.

YOUR MISSION

THE NEXT 7 DAYS

- ✓ **Day 16** — Today: teams, projects, repos
- ✓ **Day 17** — Sprint 1: Skeleton
- ✓ **Day 18** — Sprint 2: Core Logic
- ✓ **Day 19** — Sprint 3: Data
- ✓ **Day 20** — Sprint 4: Polish
- ✓ **Day 21** — Sprint 5: Freeze
- ✓ **Day 22** — Deploy (live!)

YOUR SUPPORT

WHO HELPS YOU

-  **Email** · hello@zlaark.com
-  **In-session** · raise a hand
-  **Your team** — ask them first
-  **Notes + cheatsheets**
-  **Your team leader** — coordinates

You are not alone. Ask early, ask often.

DEMO DAY

DAY 25

-  **Live demo** to faculty panel
-  Every student answers questions
-  **Certificates** for those who qualify
-  You built a real app. **From scratch.**

9 days from now, you'll show this to people who matter.